Week 1 - Weekly Assessment

Python + Software Engineering

Duration: 1 day (about 6 hours) | Mode: Individual, hands-on

Concepts Covered: All Week 1 topics: Python basics, functions (default values, *args, **kwargs), classes, inheritance, decorators, file I/O with pathlib, logging, virtual environments, type hints, REST calls, nested JSON, CSV/Excel/PDF handling, retry logic, HMAC webhook validation, environment variables, Git workflow, project structure, pre-commit, FastAPI + pytest

Scenario

Yoho's HR team needs a small Employee Data Service. You will build it in one day using everything learned this week: clean Python code, file handling, a reliable API client, a secure webhook check and a tested FastAPI service in a properly structured Git repository.

Suggested time plan: Task 1 - 60 min | Task 2 - 75 min | Task 3 - 75 min | Task 4 - 120 min | Final cleanup and PR - 30 min

Instructions

- Work individually. You may use documentation and the internet, but do not copy another trainee's work
- Commit to Git regularly with clear messages; do not commit `.env` files, keys or passwords
- Every task must run and be demonstrated; unfinished tasks are assessed only on what works
- Write short comments or a README explaining your decisions

Task - 1

Python Core and Classes

Use this data (`employees.csv`):

```
name,department,salary
Arun,Sales,30000
Priya,HR,45000
Rahul,IT,60000
Divya,IT,75000
Karthik,Finance,90000
```

Requirements

- Create `calculate_bonus(salary, rate=0.1, *extras, **options)`: the bonus is salary * rate plus the sum of the extras; if `options` contains `cap`, the bonus must not exceed it
- Create the class `Employee(name, salary)` with `get_details()` and a `@property annual_salary`
- Create `Manager(Employee)` with `team_size`, use `super().__init__()` and override `get_details()`
- Write a decorator `log_call` that prints the function name before it runs and apply it to `Manager.get_details`

- Use a `for` loop and `if/elif/else` to print the salary category (Low below 50000, Medium below 70000, High otherwise) for every employee in the data below

Expected Output

```
Arun - Low Salary
Priya - Low Salary
Rahul - Medium Salary
Divya - High Salary
Karthik - High Salary
Calling get_details
Divya - 75000 - Team Size: 5
```

Concepts: Functions, *args, **kwargs, classes, inheritance, super(), @property, decorators, control flow

Task - 2

File Handling, Types and Environment

Requirements

- Create a virtual environment and a `requirements.txt` (requests, python-dotenv, openpyxl, pymupdf, fastapi, uvicorn, pytest)
- Using `pathlib` and `csv.DictReader`, read `employees.csv`, then write `employees.json` and `high_earners.csv` (salary above 50000)
- Create `salary.xlsx` with `openpyxl` and read it back; extract the text of page 1 of any PDF with PyMuPDF and print the first 100 characters
- Write `read_file(path)` with `try/except FileNotFoundError` and `json.JSONDecodeError`; log errors with `logging.getLogger(__name__)` (no `print`)
- Add type hints to every function (`List[dict]`, `Optional[dict]`, `Union[int, str]`) and add `find_employee(name) -> Optional[dict]`

Expected Output

```
Rows read: 5 | High earners written: 3
salary.xlsx created and read back: 5 rows
ERROR - File not found: missing.txt
```

Concepts: pathlib, JSON/CSV/Excel/PDF, logging, exceptions, venv, type hints

Task - 3

API Client and Webhook Security

Requirements

- Store `API_KEY` and `WEBHOOK_SECRET` in `.env` and load them with `python-dotenv` (nothing hardcoded; `.env` must be in `.gitignore`)
- Create a `requests.Session` with `Retry(total=3, backoff_factor=1, status_forcelist=[500, 502, 503, 504])`

- GET `https://jsonplaceholder.typicode.com/users` through that session and print id, name and email of the first 5 users
- Using `.get()` only, print city, latitude, zip and company name of the first user; missing values print `Not Available`
- Write `verify_signature(payload, signature, secret)` with HMAC-SHA256 and `hmac.compare_digest`; test a valid, a tampered and a wrong-secret case

Expected Output

```
1 - Leanne Graham - Sincere@april.biz
...
City: Gwenborough | Zip: Not Available
Valid: True | Tampered: False | Wrong secret: False
```

Concepts: REST calls, retry logic, nested JSON with .get(), os.getenv, HMAC webhook validation

Task - 4

FastAPI Service, Tests and Git

Requirements

- Create the repo structure: `src/`, `tests/`, `config/`, `scripts/`, `docs/`, `.env.example`, `.gitignore`, `pyproject.toml` (black, ruff, pytest config)
- Build a FastAPI app with `GET /health`, `GET /employees` (reads `employees.json`), `GET /employees/{name}` (404 if not found) and `POST /employees` (Pydantic validation: name not empty, salary above 0)
- Write at least 6 pytest tests (success, 404, invalid salary, empty name, health, list) and show the coverage with `pytest --cov=src`
- Install pre-commit with black and ruff hooks and make the final commit pass the hooks
- Work on `feature/employee-service`, commit in small steps with clear messages, push and open a Pull Request

Expected Output

```
GET /health -> {"status": "ok"}
GET /employees/Ravi -> 404
POST salary -5 -> 422
6 passed, coverage 80%+
```

Concepts: FastAPI, Pydantic, pytest, coverage, project structure, pre-commit, Git branching and PR